

PR QA Agent

Product Specification — Automated Regression Testing for AI-Built Web Applications

Document purpose: A build-ready scope for an open-source regression testing tool. This document is intended to be passed to an implementation agent.

Core promise: Point the tool at a repository and a deployed URL. It reads the codebase, connects to the running application, asks what matters, and writes a baseline test suite that covers the product's real behavior — runnable on demand, with evidence behind every result. Once that baseline exists, it becomes the thing that stops broken code from shipping.

1. Product Definition

PR QA Agent establishes and maintains a functional regression baseline for a web application, and runs it on demand against a real deployment.

Two lifecycles:

- **Baseline establishment.** The tool analyses the repository, connects to a running deployment, interviews the developer about what must never break, and proposes a ranked inventory of tests. Approved scenarios are authored, executed against the real application, and certified as deterministic. That certified set is the baseline.
- **Regression guarding.** The baseline runs on demand today, and on every pull request once the GitHub integration lands. Failures are classified against a known-good comparison, with evidence and a suggested fix.

Primary user: a developer or small team shipping a web application built with Lovable, v0, Bolt, or by hand, deployed on Vercel or similar, typically with Supabase as the backend — and with no test suite whatsoever.

Primary objective: prevent the shipping of code that breaks working behavior. Validation of new features is a secondary, opt-in capability that activates only when the developer supplies acceptance criteria.

Distribution: open source under Apache 2.0. Implemented in Python, targeting TypeScript/React codebases. Usable as a CLI, as a Claude Code skill, and through a local web interface.

2. Target Stack

The tool is deliberately narrow. Depth on one stack beats shallow coverage of many, and the target stack is unusually regular because much of it is machine-generated from a small number of templates — which is precisely what makes accurate static analysis possible.

Layer	Supported in MVP
Frontend	React with Vite (Lovable, Bolt, v0 output) and Next.js App Router
Language	TypeScript and TSX
Backend	Supabase (Postgres, Auth, Storage, Edge Functions, RLS); Next.js Route Handlers and Server Actions
Auth	Supabase Auth, Clerk, NextAuth
Deployment	Vercel, Netlify, Lovable hosting, or a local dev server
Third-party	Stripe, Resend, OpenAI and similar SDKs — detected and substituted, not exercised

The defining characteristic of this stack: a large share of the application’s backend behavior is expressed as Supabase client calls made directly from React components. There is frequently no API layer at all. Any tool that models the product surface as “HTTP route to handler” finds nothing here.

3. Design Commitments

- **The deployment already exists.** The tool connects to a running application; it does not orchestrate containers, boot services, or reconstruct an environment. This removes the single largest source of failure in tools of this kind.
- **Tests exercise the running product through its real interface.** For this stack that is predominantly the browser, because that is where the behavior lives. API-level and database-level scenarios are used where a real seam exists.
- **The baseline encodes known-good behavior.** A scenario certified green against a working deployment asserts how the product behaved before a change. Tests derived only from changed code assert the new behavior as correct by construction and are never treated as regression evidence.
- **Every claim carries evidence.** A pass names the scenarios that ran and what they asserted. A failure carries the browser trace, screenshots, network log, console output, database state, and a reproduction command. There is no single opaque quality score.
- **Every expectation has provenance:** approved scenario, database schema or RLS policy, API schema, stated acceptance criteria, explicit developer instruction, behavior observed against a working deployment, or model inference. Inference alone never blocks.
- **Deterministic core, model at the edges.** Code analysis, execution, state management, classification, and policy are deterministic and testable without an API key. The model interprets intent, proposes scenarios, authors test code, and explains failures. Nothing else.
- **Determinism is engineered.** Seeded data, reset between scenarios, stable selectors, explicit waits on observable state, substituted third parties. A scenario that cannot be stabilised is quarantined, not admitted.
- **Human approval gates the suite.** Proposed scenarios, baseline updates, and suggested fixes are reviewed by a person. The tool never writes to a source branch and never applies a fix.
- **Destructive operations require consent.** The tool refuses to run mutating scenarios against a deployment not explicitly marked as disposable.
- **All state is human-readable and version-controlled.** Intent, surfaces, scenarios, mock contracts, and results live in a `.qa/` directory as YAML the developer owns and can edit by hand.

4. Stack Comprehension

Analysis is entirely deterministic — tree-sitter over TypeScript and TSX, plus SQL migration parsing. No model involvement, so it is fast, free, offline, and verifiable against committed golden files.

What counts as a surface

- **Screens** — React Router routes or Next.js App Router segments, and the component tree behind each.
- **Data operations** — Supabase client calls, with the table, operation, filters, and columns resolved statically: `.select()`, `.insert()`, `.update()`, `.delete()`, `.rpc()`.
- **Server endpoints** — Next.js Route Handlers, Server Actions marked `'use server'`, and Supabase Edge Functions.
- **Authorization policies** — row-level security policies parsed from `supabase/migrations/`, with the tables and roles they govern.
- **Schema** — tables, columns, constraints, foreign keys, and enums.
- **Auth boundaries** — protected routes, session guards, role checks.
- **External dependencies** — third-party SDK call sites, which become the substitution boundary.

Each surface is emitted with its source location, the screens that reach it, and a criticality prior derived from whether it writes data, sits behind authentication, or touches payments.

Output is `qa/surfaces.yaml`: a complete, inspectable map of what the application does. It is useful on its own, before any test exists.

5. Test Philosophy and the Substitution Boundary

Scenario tiers

- **Browser scenarios (primary)**. Playwright against the deployed application. For a React application talking directly to Supabase, this is the only place the behavior is observable end to end. This is the bulk of the suite, which makes selector stability and wait discipline load-bearing rather than incidental.
- **Authorization scenarios**. Generated directly from RLS policies and schema: assert that one user cannot read, modify, or delete another user's rows, and that anonymous access is refused where policy says it must be. Cheap to generate, near-impossible for these teams to write themselves, and the failure class that hurts them most.
- **Endpoint scenarios**. Direct HTTP against Route Handlers, Server Actions, and Edge Functions where they exist. Fast and stable; used wherever a real seam is available.

Real, always

- The deployed application, exactly as users receive it.
- Supabase — real Postgres, real migrations, real RLS, real Auth. Substituting the database would defeat the purpose, since authorization and data integrity are the behavior under test.
- Authentication, using seeded test users.

Substituted by policy

- Third-party APIs that cost money, cause irreversible side effects, or are nondeterministic — payments, transactional email and SMS, model APIs.
- Substitution happens at the network layer. An outbound request matching no contract is a **contract mismatch** and fails the scenario. It is never a silent pass; that is how functional suites come to lie.

State isolation

The hardest remaining problem, and the one that replaces environment orchestration as the primary technical risk. Supported strategies, in order of preference:

1. **Local Supabase** — `supabase db reset` plus seed, giving complete isolation.
2. **Database branch** — an ephemeral branch per run against a hosted project.
3. **Scoped seed and cleanup** — every scenario creates its own data under a run-scoped namespace and removes it afterwards. The only option against a shared deployment, and the reason concurrent runs must be namespaced.

6. Deployment Connection and Authentication

The tool is told about one or more endpoints. There may be several — a local dev server, a preview deployment, and production — and each carries its own safety posture.

Each endpoint declares its kind (web application, endpoint base, or edge function host), its URL, whether it is disposable, and which operations are permitted against it. Production is read-only unless explicitly and repeatedly confirmed.

Authentication

Every application in this category is behind a login wall, which makes authentication the first practical obstacle rather than a detail. It is a first-class component with an adapter per provider:

- **Supabase Auth** — seeded test users, password sign-in, or a service-role-minted session.
- **Clerk** and **NextAuth** — provider-specific sign-in flows.
- Sessions are captured once as browser storage state and reused across scenarios, with per-role sessions for authorization testing.

Live probe

With a session established, the tool explores the running application read-only: loads reachable screens, records the DOM structure and stable selector candidates, and captures the Supabase and third-party network traffic each screen produces. Mutating interactions require explicit consent.

The probe is then reconciled against the static scan: screens in code that could not be reached, queries firing that the scan did not predict, policy denials, and console errors. This reconciliation is what grounds generated scenarios in observed behavior rather than inference.

7. Baseline Establishment

A. Intent capture

A conversation, not a form. What does this product do, who uses it, what must never break, what would be worst if it silently stopped working. Answers are structured into journeys and criticality tiers, shown back for confirmation, and stored as a versioned intent document. The tool asks follow-up questions where the repository is ambiguous — an unfamiliar table, a screen it could not reach, an operation with no obvious trigger.

B. Gap map

Surfaces joined against observed behavior and any existing tests, ranked by criticality. For this stack the ranking has strong free priors: write operations, anything behind authentication, anything touching payments, and every RLS policy rank high automatically.

C. Proposed test inventory

A ranked, reviewable list. Each item states the surface it protects, the behavior to assert, the scenario in given/when/then form, the test data it needs, the dependencies to be substituted, its estimated runtime, and the provenance of the expectation. The developer approves, edits, defers, or rejects. Nothing is authored without approval.

D. Authoring

Approved items become scenario specifications in YAML — the durable artifact — which compile to runnable Playwright or endpoint test code. Keeping the specification separate from the emitted code means tests can be regenerated, retargeted, or reviewed without redoing the analysis.

E. Certification

A scenario enters the baseline only after passing a configurable number of consecutive runs, three by default. Scenarios that cannot be stabilised are quarantined with a diagnosis rather than admitted and later ignored.

F. Baseline report

Surface coverage before and after, what is now protected, what remains knowingly unprotected and why, total suite runtime, and the remaining ranked backlog. Onboarding is incremental; nobody approves two hundred scenarios on day one.

8. Running the Baseline

Runs are on demand in the MVP: the whole suite, a tagged subset, or a single scenario, against any configured endpoint.

- Results stream as each scenario completes rather than arriving as a batch at the end.
- Every failure produces an evidence bundle: browser trace, screenshot and video, network log, console output, relevant database state, and a single-command reproduction.

- Failures are classified deterministically where possible — contract mismatch, authentication failure, deployment unreachable, timeout, assertion failure — and explained by the model only once classified.
- Nondeterministic scenarios are quarantined automatically, excluded from the blocking set, and assigned a repair task.
- Per-scenario coverage is recorded from the first release, ahead of the risk-based selection that consumes it.

9. Coverage Model

Line coverage is a poor fit for a browser-driven suite against a deployed application. What is reported instead:

- **Surface coverage** — the fraction of screens, data operations, and endpoints with a passing certified scenario, weighted by criticality. The headline metric.
- **Journey coverage** — the fraction of intent-declared journeys covered end to end.
- **Policy coverage** — the fraction of RLS policies with an authorization scenario asserting them.
- **Data operation coverage** — which tables and operations are exercised, and which writes are entirely unprotected.
- **Substitution coverage** — which third-party dependencies have contracts and which are unguarded.

10. Interfaces

One engine, three front ends. The CLI is the complete interface; everything else is a view onto the same `.qa/` state, so no capability is exclusive to a single surface.

Command line

<code>qa scan [path]</code>	Analyse the repository — screens, data operations, endpoints, policies, schema. No API key required.
<code>qa surfaces</code>	List and filter what the scan found.
<code>qa init</code>	Configure entrypoints, authentication, safety posture.
<code>qa probe</code>	Explore the running deployment and reconcile with the scan.
<code>qa intent</code>	Capture what the product does and what must never break.
<code>qa gaps</code>	Ranked report of the tests this application is missing.
<code>qa propose</code>	Draft scenarios for review.
<code>qa approve <id>...</code>	Accept proposals into the baseline.
<code>qa generate</code>	Emit runnable test code from approved specifications.
<code>qa run</code>	Execute the suite, a tag, or a single scenario.
<code>qa certify</code>	Promote scenarios that pass N consecutive runs.
<code>qa report</code>	Results and evidence from the last run.
<code>qa web</code>	Start the local web interface.
<code>qa providers</code>	Configure and inspect model providers.
<code>qa doctor</code>	Diagnose configuration, connectivity, and auth.

Every command is non-interactive when given complete arguments and interactive when not, so the same tool serves a human at a terminal and an agent in a pipeline. Machine-readable output is available on every command via `--json`.

Claude Code skill

The repository ships a skill that teaches Claude Code to drive the CLI: when to scan, how to interpret a gap map, how to propose and approve scenarios, how to read an evidence bundle, and which operations require human confirmation. Because every command has a JSON mode and the entire state is plain YAML on disk, an agent can operate the tool end to end without screen-scraping terminal output.

Web interface

`qa web` serves a local application over the same `.qa/` directory — no separate database, no cloud dependency, no divergence between what the CLI and the browser believe. It exists for the parts that are genuinely better with a screen: reviewing the surface map, working through the gap map, approving proposed scenarios, watching a run stream, and inspecting an evidence bundle with its screenshots, video, and network log.

11. Model Providers

Model access is provider-agnostic behind a single gateway. Three providers are supported at launch:

Provider	Notes
Anthropic	Claude models via the native API.
OpenAI	GPT models via the OpenAI API.
Kimi	Moonshot Kimi models via an OpenAI-compatible endpoint with a configurable base URL, which also covers self-hosted and foundry deployments.

Provider and model are configured per repository and overridable per command. The gateway is the only component permitted to make model calls, which keeps three properties true: prompts live in versioned files rather than inline in business logic, every request and response is recorded to disk as a cassette, and the entire test suite of the tool itself runs offline against recorded cassettes with no API key.

12. Repository Configuration

```
version: 1

project:
  framework: vite-react          # vite-react | nextjs-app
  backend:   supabase
  auth:      supabase

entrypoints:
- name:      local
  kind:      web
  url:       http://localhost:5173
  disposable: true
- name:      preview
  kind:      web
  url:       https://myapp-git-branch.vercel.app
  disposable: true
- name:      production
  kind:      web
  url:       https://myapp.com
  disposable: false
  allow:     [read]              # mutating scenarios refused

supabase:
  url:       env:SUPABASE_URL
  anon_key:  env:SUPABASE_ANON_KEY
  migrations: supabase/migrations
  isolation: local_reset        # local_reset | branch | scoped_seed

test_users:
- { role: owner, email: env:QA_OWNER_EMAIL, password: env:QA_OWNER_PW }
- { role: member, email: env:QA_MEMBER_EMAIL, password: env:QA_MEMBER_PW }

substitution:
  network:      deny
  contracts:    .qa/contracts/
  on_unmatched_request: fail
  external:
    - { name: stripe, match: api.stripe.com }
    - { name: resend, match: api.resend.com }

model:
  provider: kimi                 # anthropic | openai | kimi
  model:    kimi-k2
  base_url: env:KIMI_BASE_URL
  api_key:  env:KIMI_API_KEY

run:
  certification_runs: 3
  parallel:           4
  retries:            0
  timeout_seconds:    60
```


13. Security

- Production entrypoints are read-only by default. Mutating scenarios against a deployment not marked disposable are refused, not warned about.
- Credentials are referenced by environment variable, never stored in `.qa/`, and redacted from logs, traces, screenshots, network captures, and model prompts.
- Test users are seeded and scoped. Service-role keys are used only for seeding and reset, never exposed to a browser context.
- Substituting third-party dependencies is also a security boundary: test runs cannot reach a real payment processor or mail provider.
- The model never controls shell execution, credential retrieval, repository writes, or destructive database operations. It proposes; deterministic components act.
- Nothing is transmitted anywhere except to the configured model provider. There is no telemetry unless explicitly enabled.

14. Components

Component	Responsibility	Model
Scanner	Parses the repository into screens, data operations, endpoints, policies, and schema.	No
Connector	Validates entrypoints, establishes authenticated sessions per role.	No
Prober	Explores the running application, captures DOM and network behavior.	No
Reconciler	Compares static analysis against observed behavior; reports divergence.	No
Interviewer	Captures product intent and resolves ambiguity through questions.	Yes
Gap analyser	Ranks unprotected surfaces by criticality.	No
Proposer	Drafts scenario specifications from gaps and observations.	Yes
Emitter	Compiles specifications into runnable test code.	Partial
Runner	Executes scenarios, streams events, collects evidence.	No
Isolator	Manages database state, seeding, and reset between scenarios.	No
Substitution proxy	Intercepts outbound traffic and matches it against contracts.	No
Classifier	Categorises failures deterministically; the model explains, never decides.	Partial
Curator	Certification, quarantine, drift detection, suite cost.	No
Gateway	The sole path to a model provider. Records cassettes.	—

15. Out of Scope

- Pull-request integration in the MVP. It is the immediate next milestone, and the data model anticipates it.
- Any write access to the source branch, and automatic application of suggested fixes.
- Automatic test healing or baseline updates without human approval.

- Stacks outside the target in section 2. The adapter interfaces are public so others can extend it.
- Load testing, penetration testing, mobile-device testing, and visual-regression diffing.
- Complete browser and device matrices.
- Hosted or multi-tenant operation. The tool runs on a developer machine or in the user's own CI.

16. Implementation Plan

Phase 0 — Foundation

Repository, licensing, and continuous integration. The domain model and the three adapter protocols — framework, authentication, emitter — documented as the contract contributors code against. The model gateway with all three providers and cassette record and replay. Two fixture applications used as both demonstration and integration test target. CLI skeleton and the Claude Code skill.

Phase 1 — Stack comprehension

tree-sitter analysis of TypeScript and TSX. Screens, Supabase data operations, Route Handlers, Server Actions, Edge Functions, RLS policies, schema, auth boundaries, third-party call sites. Golden-file tested against both fixtures. Delivers `qa scan`, which is useful before any test exists.

Phase 2 — Deployment connection

Entrypoint configuration and safety posture. Authentication adapters for Supabase Auth, Clerk, and NextAuth, with session capture and per-role reuse. Live probe and reconciliation against the scan. Delivers `qa init`, `qa probe`, and `qa doctor`.

Phase 3 — Intent and gaps

Conversational intent capture with ambiguity resolution, criticality ranking, and the gap map. Delivers `qa intent` and `qa gaps` — a ranked, reasoned inventory of the tests the application is missing.

Phase 4 — Scenario authoring

Scenario specifications, the review and approval loop, the Playwright emitter, the endpoint emitter, and the RLS authorization emitter. Selector strategy, including proposing test identifiers where the DOM offers nothing stable. Delivers `qa propose`, `qa approve`, and `qa generate`.

Phase 5 — Execution

The runner with streaming results and evidence collection. State isolation across all three strategies. The substitution proxy. Deterministic failure classification, certification, and quarantine. Per-scenario coverage recording. Delivers `qa run`, `qa certify`, and `qa report` — and completes the MVP.

Phase 6 — Web interface and release

The local web application over the same state, documentation, a quickstart that works in under ten minutes, and the first public release.

17. Definition of Done

The MVP is complete when a developer who has never seen the tool can:

1. Point it at a local repository and receive an accurate map of every screen, data operation, endpoint, and policy in their application.
2. Give it a deployed URL and test credentials, and have it authenticate and explore the running application.
3. Answer a short conversation about what matters, and receive a ranked list of the tests they are missing with a reason for each.
4. Approve a subset and receive working Playwright and authorization tests that pass against their real deployment.
5. Run the suite on demand and receive streamed results, with an evidence bundle and a reproduction command behind every failure.
6. Do all of the above from the command line, from Claude Code, or from a local web interface, against the model provider of their choice.

18. Next Milestone: Pull-Request Guarding

Deferred but designed for. The per-scenario coverage index recorded from the first release becomes the basis for risk-ranked selection, so the scenarios touching changed code run first. Vercel preview deployments remove the need to construct an environment for each pull request — the deployment already exists — leaving data isolation between concurrent runs as the problem to solve. Every failure is re-executed against a known-good deployment before it is called a regression. A deterministic policy engine converts results into a check status; the model recommends and explains, but never decides.

19. Product Principle

Optimize for trustworthy results, not maximum autonomy. A small suite that reliably protects the handful of behaviors an application depends on — and proves, with evidence, when one of them breaks — is worth more than a large one that generates uncertain tests nobody believes.

The measure of success is regressions caught before shipping, with a false-positive rate low enough that the developer never considers turning it off.